

ЕТАП 8 ОТ 11 · РЪКОВОДСТВО

Widget-и الأذان

Следващата молитва и колко остава – направо на началния екран. Три widget-а за Android, малък и среден за iPhone и четири вида за заключения екран на iPhone. Фонът е градиентът на текущата молитва, а оставащото време брои самата система – без да буди приложението и без да хаби батерия.

Kotlin · RemoteViews

Chronometer

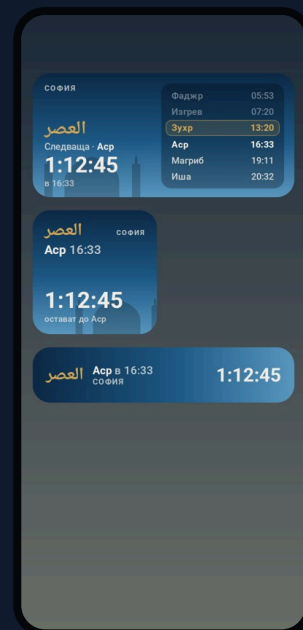
expo-widgets

@expo/ui · SwiftUI

Нова икона

Jest – 226 теста

20 файла



Какво е решено в този етап

Android: „Следваща“ 2×2, „Денят“ 4×2, „Лента“ 4×1 – собствен Kotlin, без библиотека · iPhone: expo-widgets – малък, среден, над часа, кръгъл (дъга), кръгъл (арабско име), правоъгълен · оставащото време е винаги Ч:ММ:СС – брои го системата · кадри за 7 дни напред · шрифтовете са системните (widget-ите не могат да ползват свои) · иконата – злато „ا“ върху нощно синьо, като в тоскур-а.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нови пакети, икона и нов build

03 Кадрите

04 Android – Kotlin

05 Android – изгледите

06 iPhone – SwiftUI

07 Тестове

08 Проверка

Стъпка 1

Какво прави етапът

Widget-ът е „снимка“, която системата държи на началния екран. Приложението не може да я рисува всяка секунда – затова:

- **Оставащото време** е вграденият брояч на системата – `Chronometer` на Android, `Text` с таймер на iPhone. Казваме му „брой до 16:33“ и той сменя цифрите сам, всяка секунда, без приложението.
- **Всичко останало** (молитвата, фонът, списъкът за деня) приложението го изчислява предварително – „кадри“ за 7 дни: нов кадър в часа на всяка молитва и в полунощ.
- **Android** сменя кадъра с `AlarmManager` в часа на молитвата (без да буди телефона); **iPhone** – сам, по `timeline` на `WidgetKit`.

Дърво на проекта

```
Приложението (при стартиране, връщане, промяна на настройките, фоновата задача)
├── updateWidgets() ─────────────────── src/services/widgets.ts
└──
    └── planWidget() – кадри за 7 дни ─── src/domain/widget.ts
        ├── Android → setWidgetData(JSON) ─── src/widgets/index.ts
        │   ├── WidgetStore (SharedPreferences) ─── modules/adhan-native/.../WidgetStore.kt
        │   ├── PrayerWidgets – рисува, планира ─── modules/adhan-native/.../PrayerWidgets.kt
        │   └── следващото обновяване (AlarmManager RTC)
        └── iPhone → updateTimeline() ─── src/widgets/index.ios.ts
            ├── PrayerWidget / PrayerArabicWidget ─── src/widgets/PrayerWidget.tsx (SwiftUI)
```

Ако приложението не се отваря 7 дни (и фоновата задача не е минала), widget-ът показва „Отвори Езан, за да се покажат часовете.“

Стъпка 2

Нови пакети, икона и нов build

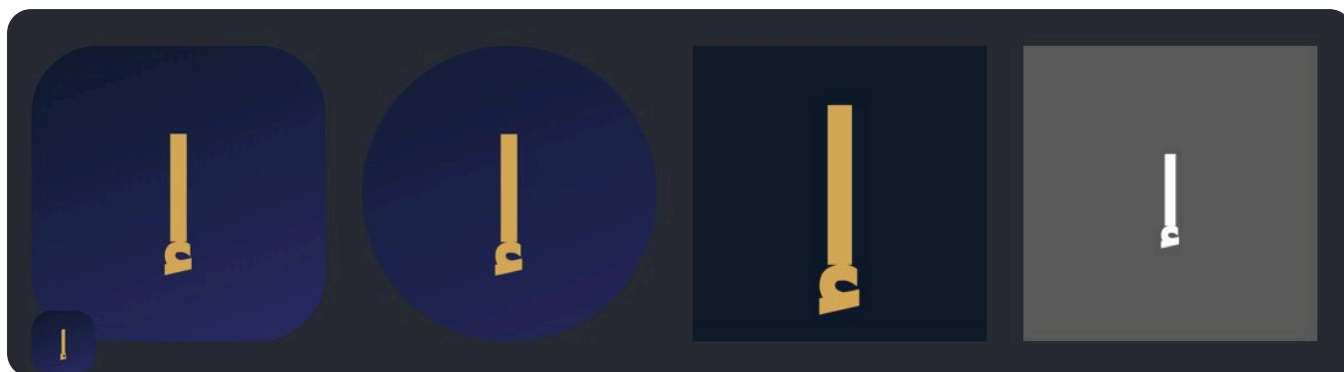
Два нови пакета (само за iPhone), нов native код за Android и нова икона – нужен е `prebuild` и нов `build`.

Инсталиране, `prebuild` и нов `build`

```
npx expo install expo-widgets @expo/ui
npx expo prebuild --platform android --clean
npx expo run:android
```

- **expo-widgets** – официалният модул на Expo за widget-и на iPhone (`WidgetKit`). Widget-ът се пише на TypeScript с компонентите на `@expo/ui` (SwiftUI).
- На Android **expo-widgets** е изключен от `autolinking` (`package.json` → `expo.autolinking.android.exclude`) – Android частта му още е празна; widget-ите за Android са наш `Kotlin`. `@expo/ui` остава и на Android: щом е инсталиран, `expo-router` го ползва за лентата на `Stack` – без него приложението спира с „Cannot find native module 'ExpoUI“.
- **Иконата** е сменена: `assets/icon.png`, `adaptive` (`android-icon-*.png` – фон, буква, монохромна за темите на Android 13+), `splash-icon.png` и `favicon.png`.

- **iPhone:** prebuild създава разширението `ExpoWidgetsTarget` и App Group `group.com.ilkoadamov.adhan` (през него приложението подава кадрите).



Иконата: iPhone · Android (adaptive) · splash · монохромна

Стъпка 3

Кадрите

Чиста логика – една и съща за двете системи, с тестове.

widget.ts src/domain

- `widgetEntryAt` – какво показва widget-ът в даден момент: текущата молитва (фон), следващата (име, арабско, час, кога е), текстовете („в 16:33“, „остават до Аср“) и шестте реда за деня.
- `planWidget` – кадрите от „сега“ до 7 дни напред. Първият започва от началото на текущата молитва, затова в рамките на една молитва резултатът е един и същ – приложението не обновява widget-a напразно.

src/domain/widget.ts

160 реда

```
import { formatHM } from './format';
import { getSchedule, PRAYERS, type PrayerId } from './prayers';
import { computeThreeDays, type TimesOptions } from './times';

/**
 * Widget-ите (етап 8) – какво показват във всеки момент.
 *
 * Приложението изчислява предварително „кадрите“ за следващите дни: нов кадър в часа на
 * всяка молитва (сменят се следващата молитва и фонът) и в полунощ (сменя се денят в списъка).
 * Android (Kotlin) и iPhone (WidgetKit) само показват кадъра, който важи в момента, а
 * оставащото време брои сама системата (Chronometer / Text с таймер) – без да буди приложението.
 */

/** Колко дни напред – widget-ът работи толкова, дори приложението да не се отваря. */
export const WIDGET_DAYS = 7;

export type WidgetRowState = 'past' | 'now' | 'next' | 'later';

export interface WidgetRow {
  id: PrayerId;
  name: string;
  time: string;
  state: WidgetRowState;
}
```

```

export interface WidgetEntry {
  /** От кога важи кадърът (ms). */
  from: number;
  /** Текущата молитва - по нея е фонът. */
  phase: PrayerId;
  /** Фонът: горе → среда (55%) → долу. */
  colors: readonly [string, string, string];
  /** „СОФИЯ“ */
  place: string;
  next: PrayerId;
  /** „Асп“ */
  nextName: string;
  /** „العصر“ */
  nextArabic: string;
  /** „16:33“ */
  nextTime: string;
  /** Часът на следващата молитва (ms) - до него брой оставащото време. */
  nextAt: number;
  /** Началото на текущата молитва (ms) - за дъгата на заключения екран. */
  prevAt: number;
  /** „Следваща“ */
  nextLabel: string;
  /** „в 16:33“ / „утре в 05:54“ */
  atText: string;
  /** „16:33“ / „утре 05:54“ */
  shortAt: string;
  /** „остават до Асп“ */
  untilText: string;
  /** „остават“ */
  leftLabel: string;
  /** Шестте часа за деня. */
  rows: WidgetRow[];
}

export interface WidgetTexts {
  prayers: Record<PrayerId, string>;
  next: string;
  at: (time: string) => string;
  tomorrowAt: (time: string) => string;
  tomorrowShort: (time: string) => string;
  until: (name: string) => string;
  left: string;
}

export interface WidgetInput {
  now: Date;
  options: TimesOptions;
  placeName: string;
  texts: WidgetTexts;
  gradients: Record<PrayerId, readonly [string, string, string]>;
  days?: number;
}

const dayAt = (d: Date, offset: number) => new Date(d.getFullYear(), d.getMonth(), d.getDate() + offset);

/** Кадърът в момента `at`. */
export function widgetEntryAt(at: Date, input: Omit<WidgetInput, 'now' | 'days'>): WidgetEntry {
  const { texts } = input;
  const s = getSchedule(at, computeThreeDays(at, input.options));
  const nextTime = formatHM(s.next.time);
  const nextName = texts.prayers[s.next.id];

```

```

return {
  from: at.getTime(),
  phase: s.current.id,
  colors: input.gradients[s.current.id],
  place: input.placeName.toLocaleUpperCase(),
  next: s.next.id,
  nextName,
  nextArabic: PRAYERS[s.next.id].arabic,
  nextTime,
  nextAt: s.next.time.getTime(),
  prevAt: s.current.time.getTime(),
  nextLabel: texts.next,
  atText: s.isNextTomorrow ? texts.tomorrowAt(nextTime) : texts.at(nextTime),
  shortAt: s.isNextTomorrow ? texts.tomorrowShort(nextTime) : nextTime,
  untilText: texts.until(nextName),
  leftLabel: texts.left,
  rows: s.today.map((p) => {
    const state = s.rowState(p.id);
    return {
      id: p.id,
      name: texts.prayers[p.id],
      time: formatHM(p.time),
      state:
        state === 'now'
          ? 'now'
          : state === 'past'
            ? 'past'
            : !s.isNextTomorrow && p.id === s.next.id
              ? 'next'
              : 'later',
    };
  }),
};
}

/**
 * Кадрите от „сега“ до `days` дни напред. Първият кадър започва от последната смяна преди
 * `now` (началото на текущата молитва или полунощ), затова при едни и същи настройки
 * резултатът е един и същ през целия интервал – приложението не обновява widget-а напразно.
 */
export function planWidget(input: WidgetInput): WidgetEntry[] {
  const { now, options } = input;
  const days = input.days ?? WIDGET_DAYS;
  const t = now.getTime();

  // всички моменти на смяна: часовете на молитвите и полунощ, от вчера до `days` дни напред
  const changes = new Set<number>();
  for (let i = -1; i <= days; i++) {
    const day = dayAt(now, i);
    changes.add(day.getTime());
    const three = computeThreeDays(day, options);
    for (const p of three.today) changes.add(p.time.getTime());
  }
  const sorted = [...changes].sort((a, b) => a - b);
  const end = dayAt(now, days).getTime();
  const firstIdx = Math.max(0, sorted.findIndex((x) => x > t) - 1);
  const moments = sorted.slice(firstIdx).filter((x, i) => i === 0 || x <= end);

  const out: WidgetEntry[] = [];
  for (const m of moments) {
    const e = widgetEntryAt(new Date(m), input);
  }
}

```

```

    // полунощ без промяна в нищо видимо – не е нужен нов кадър (напр. ако часовете са невалидни)
    const prev = out[out.length - 1];
    if (prev && sameView(prev, e)) continue;
    out.push(e);
  }
  return out;
}

function sameView(a: WidgetEntry, b: WidgetEntry): boolean {
  const { from: _a, ...ra } = a;
  const { from: _b, ...rb } = b;
  return JSON.stringify(ra) === JSON.stringify(rb);
}

```

widgets.ts src/services

- Взема настройките и езика, изчислява кадрите и ги дава на Android или iPhone – само ако са се променили.

src/services/widgets.ts

37 реда

```

import { Platform } from 'react-native';

import { planWidget } from '@domain/widget';
import { getI18n } from '@i18n';
import { selectLocation, selectTimesOptions, useSettings } from '@store/settings';
import { PHASE_GRADIENTS } from '@theme/gradients';
import { pushWidgetEntries } from '@widgets';

/**
 * Обновява widget-ите (етап 8): изчислява кадрите за следващите 7 дни и ги дава на
 * Android (native) или на iPhone (WidgetKit timeline).
 * Вика се при стартиране, при връщане в приложението, при промяна на настройките и от
 * фоновата задача. Ако нищо не се е променило – не прави нищо (кадрите са едни и същи
 * през целия интервал на една молитва, виж planWidget).
 */
let lastSignature = '';

export function updateWidgets(now: Date = new Date()): void {
  if (Platform.OS !== 'android' && Platform.OS !== 'ios') return;
  try {
    const s = useSettings.getState();
    const { lang, t } = getI18n();
    const entries = planWidget({
      now,
      options: selectTimesOptions(s).options,
      placeName: selectLocation(s).names[lang],
      texts: { prayers: t.prayers, ...t.hero, ...t.widget },
      gradients: PHASE_GRADIENTS,
    });
    const signature = JSON.stringify(entries);
    if (signature === lastSignature) return;
    pushWidgetEntries(entries, t.widget.empty);
    lastSignature = signature;
  } catch (e) {
    console.warn(['widgets', e]);
  }
}

```

index.ts src/widgets

- Android: JSON-ът отива в native частта.

src/widgets/index.ts

11 реда

```
import type { WidgetEntry } from '@domain/widget';

import { setWidgetData } from '../../modules/adhan-native';

/**
 * Android: кадрите отиват в native частта (modules/adhan-native - WidgetStore.kt), която
 * рисува widget-ите и ги обновява в часа на всяка молитва. iPhone - виж index.ios.ts.
 */
export function pushWidgetEntries(entries: WidgetEntry[], emptyText: string): void {
  setWidgetData(JSON.stringify({ v: 1, emptyText, entries }));
}
```

index.ios.ts src/widgets

- iPhone: кадрите стават timeline; след последния – „Отвори Езан...“.

src/widgets/index.ios.ts

24 реда

```
import type { WidgetEntry } from '@domain/widget';

import { prayerArabicWidget, prayerWidget, type PrayerWidgetProps } from './PrayerWidget';

/**
 * iPhone: кадрите стават timeline на WidgetKit - iOS сам сменя кадъра в часа на всяка
 * молитва и в полунощ. След последния кадър (ако приложението не се отваря 7 дни) -
 * „Отвори Езан...“.
 */
export function pushWidgetEntries(entries: WidgetEntry[], emptyText: string): void {
  if (!entries.length) {
    const empty: PrayerWidgetProps = { empty: true, emptyText };
    prayerWidget.updateSnapshot(empty);
    prayerArabicWidget.updateSnapshot(empty);
    return;
  }
  const last = entries[entries.length - 1];
  const timeline: { date: Date; props: PrayerWidgetProps }[] = [
    ...entries.map((e) => ({ date: new Date(e.from), props: e })),
    { date: new Date(last.nextAt), props: { empty: true, emptyText } },
  ];
  prayerWidget.updateTimeline(timeline);
  prayerArabicWidget.updateTimeline(timeline);
}
```

useNotificationScheduler.ts src/hooks

- updateWidgets() при стартиране, при връщане в приложението и при промяна на настройките.

src/hooks/useNotificationScheduler.ts

63 реда

```
import { useEffect } from 'react';
import { AppState } from 'react-native';

import { registerBackgroundReschedule } from '@services/backgroundTask';
```

```

import { refreshPermission, requestPermission, rescheduleNotifications } from '@services/notifications';
import { updateWidgets } from '@services/widgets';
import { useAlertPrefs } from '@store/alertPrefs';
import { useSettings } from '@store/settings';
import { useSounds } from '@store/sounds';

/**
 * Пуска се веднъж в корена на приложението, след като настройките са заредени.
 * - при първо пускане пита за разрешение за известия;
 * - планира известията и ги пренасрочва при всяка промяна на мястото, часовете,
 * камбанките, звуците, напомнянето и вибрацията (с малко изчакване, за да не се планира
 * при всяко натискане на „+“);
 * - при връщане в приложението (и след „Аларми и напомняния“ в настройките на телефона);
 * - регистрира фоновата задача;
 * - обновява widget-ите (етап 8) – при стартиране, при промяна на настройките и при връщане.
 */
export function useNotificationScheduler(ready: boolean) {
  useEffect(() => {
    if (!ready) return;

    updateWidgets();
    (async () => {
      const permission = await refreshPermission();
      if (permission === 'undetermined') await requestPermission();
      await rescheduleNotifications();
      await registerBackgroundReschedule();
    })();

    let timer: ReturnType<typeof setTimeout> | undefined;
    const soon = () => {
      clearTimeout(timer);
      timer = setTimeout(rescheduleNotifications, 600);
    };
    let widgetTimer: ReturnType<typeof setTimeout> | undefined;
    const unsubWidgets = useSettings.subscribe(() => {
      clearTimeout(widgetTimer);
      widgetTimer = setTimeout(() => updateWidgets(), 600);
    });
    const unsubSettings = useSettings.subscribe(soon);
    const unsubAlerts = useAlertPrefs.subscribe(soon);
    const unsubSounds = useSounds.subscribe(soon);
    const sub = AppState.addEventListener('change', (state) => {
      if (state === 'active') {
        updateWidgets();
        rescheduleNotifications();
      }
    });

    return () => {
      clearTimeout(timer);
      clearTimeout(widgetTimer);
      unsubWidgets();
      unsubSettings();
      unsubAlerts();
      unsubSounds();
      sub.remove();
    };
  }, [ready]);
}

```

backgroundTask.ts src/services

- Фоновата задача измества кадрите напред (винаги 7 дни).

src/services/backgroundTask.ts

52 реда

```
import * as BackgroundTask from 'expo-background-task';
import * as TaskManager from 'expo-task-manager';
import { Platform } from 'react-native';

import { useAlertPrefs } from '@/store/alertPrefs';
import { useSettings } from '@/store/settings';
import { useSounds } from '@/store/sounds';

import { rescheduleNotifications } from './notifications';
import { updateWidgets } from './widgets';

/**
 * Фонова задача: веднъж на няколко часа допланира известията (и кадрите на widget-ите)
 * за следващите дни,
 * дори ако приложението не се отваря. Android – WorkManager (работи и без Google услуги),
 * iPhone – BGTaskScheduler (системата решава кога; обикновено нощем, на зарядно).
 */
export const RESCHEDULE_TASK = 'ezan-reschedule-notifications';

// Задачата трябва да е дефинирана при зареждане на кода (не в компонент) –
// системата може да пусне приложението само за нея, без да показва екран.
if (Platform.OS !== 'web') {
  TaskManager.defineTask(RESCHEDULE_TASK, async () => {
    try {
      // при пускане само за задачата настройките още не са прочетени от паметта
      await Promise.all([
        useSettings.persist.rehydrate(),
        useAlertPrefs.persist.rehydrate(),
        useSounds.persist.rehydrate(),
      ]);
      // widget-ите – кадрите се изместват напред (винаги 7 дни)
      updateWidgets();
      await rescheduleNotifications();
      return BackgroundTask.BackgroundTaskResult.Success;
    } catch {
      return BackgroundTask.BackgroundTaskResult.Failed;
    }
  });
}

export async function registerBackgroundReschedule(): Promise<void> {
  if (Platform.OS === 'web') return;
  try {
    const status = await BackgroundTask.getStatusAsync();
    if (status !== BackgroundTask.BackgroundTaskStatus.Available) return;
    if (!(await TaskManager.isTaskRegisteredAsync(RESCHEDULE_TASK))) {
      await BackgroundTask.registerTaskAsync(RESCHEDULE_TASK, { minimumInterval: 6 * 60 }); // минути
    }
  } catch (e) {
    console.warn('[background-task]', e);
  }
}
```

Стъпка 4

Android – Kotlin

Три AppWidgetProvider-а и един общ код за рисуване.

WidgetStore.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Пази кадрите в SharedPreferences (има ги и след рестарт); избира кадъра, който важи сега, и кога е следващата смяна.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/WidgetStore.kt

99 реда

```
package expo.modules.adhannative

import android.content.Context
import org.json.JSONObject

/**
 * Кадрите на widget-ите (етап 8) – както ги изчислява приложението (src/domain/widget.ts):
 * нов кадър в часа на всяка молитва и в полунощ, за 7 дни напред. Пазят се в
 * SharedPreferences, за да ги има и когато приложението е затворено или след рестарт.
 */
object WidgetStore {
    private const val PREFS = "ezan_widgets"
    private const val KEY = "data"

    data class Row(val name: String, val time: String, val state: String)

    data class Entry(
        val from: Long,
        val phase: String,
        val place: String,
        val nextName: String,
        val nextArabic: String,
        val nextTime: String,
        val nextAt: Long,
        val prevAt: Long,
        val nextLabel: String,
        val atText: String,
        val shortAt: String,
        val untilText: String,
        val rows: List<Row>,
    )

    data class Data(val emptyText: String?, val entries: List<Entry>)

    private fun prefs(context: Context) = context.getSharedPreferences(PREFS, Context.MODE_PRIVATE)

    /** Записва новите кадри. false – JSON-ът е същият като досега (няма нужда от обновяване). */
    fun save(context: Context, json: String): Boolean {
        val p = prefs(context)
        if (p.getString(KEY, null) == json) return false
        p.edit().putString(KEY, json).apply()
        return true
    }

    fun load(context: Context): Data {
        val json = prefs(context).getString(KEY, null) ?: return Data(null, emptyList())
        return try {
            val o = JSONObject(json)
            val arr = o.optJSONArray("entries")
        } catch (e: Exception) {
            return Data(null, emptyList())
        }
    }
}
```

```

val entries = mutableListOf<Entry>()
if (arr != null) {
    for (i in 0 until arr.length()) {
        val e = arr.optJSONObject(i) ?: continue
        val rowsArr = e.optJSONArray("rows")
        val rows = mutableListOf<Row>()
        if (rowsArr != null) {
            for (j in 0 until rowsArr.length()) {
                val r = rowsArr.optJSONObject(j) ?: continue
                rows.add(Row(r.optString("name"), r.optString("time"), r.optString("state")))
            }
        }
        entries.add(
            Entry(
                from = e.optLong("from"),
                phase = e.optString("phase"),
                place = e.optString("place"),
                nextName = e.optString("nextName"),
                nextArabic = e.optString("nextArabic"),
                nextTime = e.optString("nextTime"),
                nextAt = e.optLong("nextAt"),
                prevAt = e.optLong("prevAt"),
                nextLabel = e.optString("nextLabel"),
                atText = e.optString("atText"),
                shortAt = e.optString("shortAt"),
                untilText = e.optString("untilText"),
                rows = rows,
            ),
        )
    }
}
Data(o.optString("emptyText").isEmpty { null }, entries.sortedBy { it.from })
} catch (e: Exception) {
    Data(null, emptyList())
}
}

/** Кадърът, който важи в момента `now`; null - няма кадри или са свършили (7 дни без приложението). */
fun current(data: Data, now: Long): Entry? {
    val e = data.entries.lastOrNull { it.from <= now } ?: data.entries.firstOrNull() ?: return null
    return if (now >= e.nextAt) null else e
}

/** Кога трябва следващото обновяване: следващият кадър (или края на текущия). */
fun nextChange(data: Data, now: Long): Long? {
    val next = data.entries.firstOrNull { it.from > now }?.from
    val end = current(data, now)?.nextAt
    return listOfNotNull(next, end).minOrNull()
}
}

```

PrayerWidgets.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- **render** – фонът на молитвата, арабското име, Chronometer до часа на молитвата (**setChronometerCountDown**), редовете за деня (текущата – злато, следващата – удебелена, минали – бледи).
- **scheduleNext** – AlarmManager **RTC** в часа на следващата смяна: не буди телефона; ако спи, обновяването идва, щом екранът светне. Без „Alarms & reminders“ – **setWindow** (до ~10 мин.).

- Тап върху widget-а отваря приложението.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/PrayerWidgets.kt

308 реда

```
package expo.modules.adhannative

import android.app.AlarmManager
import android.app.PendingIntent
import android.appwidget.AppWidgetManager
import android.appwidget.AppWidgetProvider
import android.content.BroadcastReceiver
import android.content.ComponentName
import android.content.Context
import android.content.Intent
import android.graphics.Typeface
import android.os.Build
import android.os.SystemClock
import android.text.SpannableString
import android.text.Spanned
import android.text.style.StyleSpan
import android.util.Log
import android.view.View
import android.widget.RemoteViews

/**
 * Widget-ите на Езан за Android (етап 8) – по одобрения mockup:
 * - „Следваща“ 2×2 (NextPrayerWidget), „Денят“ 4×2 (DayWidget), „Лента“ 4×1 (BarWidget).
 *
 * Как се обновяват, без да хаят батерия:
 * - оставащото време е Chronometer – брои го системата всяка секунда, без приложението;
 * - в часа на всяка молитва и в полунощ AlarmManager (RTC, без събуждане на телефона) вика
 *   WidgetRefreshReceiver – той показва новия кадър (нова молитва, нов фон);
 * - ако телефонът спи, обновяването става, щом екранът светне;
 * - и при рестарт, смяна на часа/часовата зона, обновяване на приложението (BootReceiver)
 * - и всеки път, когато приложението даде нови кадри (setWidgetData).
 */
object PrayerWidgets {
    private const val TAG = "AdhanWidget"
    private const val REFRESH_REQUEST = 0xEA0108
    private const val OPEN_REQUEST = 0xEA0109

    private val PROVIDERS = listOf(
        NextPrayerWidget::class.java,
        DayWidget::class.java,
        BarWidget::class.java,
    )

    private val GOLD = 0xFFD4A857.toInt()
    private val TEXT = 0xFFF2EFE8.toInt()
    private val DIM = 0xC7F2EFE8.toInt() // 78%
    private val PAST = 0x73F2EFE8 // 45%
    private val LATER = 0xD9F2EFE8.toInt() // 85%

    private val ROW_IDS = arrayOf(
        intArrayOf(R.id.ezan_row0, R.id.ezan_row0_name, R.id.ezan_row0_time),
        intArrayOf(R.id.ezan_row1, R.id.ezan_row1_name, R.id.ezan_row1_time),
        intArrayOf(R.id.ezan_row2, R.id.ezan_row2_name, R.id.ezan_row2_time),
        intArrayOf(R.id.ezan_row3, R.id.ezan_row3_name, R.id.ezan_row3_time),
        intArrayOf(R.id.ezan_row4, R.id.ezan_row4_name, R.id.ezan_row4_time),
        intArrayOf(R.id.ezan_row5, R.id.ezan_row5_name, R.id.ezan_row5_time),
    )
}
```

```

/** Колко widget-а на Езан има на началния екран. */
fun count(context: Context): Int {
    val manager = AppWidgetManager.getInstance(context) ?: return 0
    return PROVIDERS.sumOf { manager.getAppWidgetIds(ComponentName(context, it)).size }
}

/** Новите кадри от приложението: записва ги и прерисува widget-ите, ако има промяна. */
fun setData(context: Context, json: String) {
    if (WidgetStore.save(context, json)) updateAll(context) else scheduleNext(context)
}

/** Прерисува всички widget-и на Езан и планира следващото обновяване. */
fun updateAll(context: Context) {
    try {
        val manager = AppWidgetManager.getInstance(context) ?: return
        for (cls in PROVIDERS) {
            val ids = manager.getAppWidgetIds(ComponentName(context, cls))
            if (ids.isNotEmpty()) update(context, manager, ids, kindOf(cls))
        }
    } catch (e: Exception) {
        Log.w(TAG, "update: ${e.message}")
    }
    scheduleNext(context)
}

enum class Kind { NEXT, DAY, BAR }

private fun kindOf(cls: Class<*>): Kind = when (cls) {
    DayWidget::class.java -> Kind.DAY
    BarWidget::class.java -> Kind.BAR
    else -> Kind.NEXT
}

fun update(context: Context, manager: AppWidgetManager, ids: IntArray, kind: Kind) {
    val data = WidgetStore.load(context)
    val now = System.currentTimeMillis()
    val entry = WidgetStore.current(data, now)
    val views = when (kind) {
        Kind.NEXT -> RemoteViews(context.packageName, R.layout.ezan_widget_next)
        Kind.DAY -> RemoteViews(context.packageName, R.layout.ezan_widget_day)
        Kind.BAR -> RemoteViews(context.packageName, R.layout.ezan_widget_bar)
    }
    if (entry == null) {
        renderEmpty(context, views, kind, data.emptyText)
    } else {
        render(context, views, kind, entry, now)
    }
    launchIntent(context)?.let { views.setOnClickPendingIntent(R.id.ezan_widget_root, it) }
    for (id in ids) manager.updateAppWidget(id, views)
}

private fun render(context: Context, v: RemoteViews, kind: Kind, e: WidgetStore.Entry, now: Long) {
    v.setInt(R.id.ezan_widget_root, "setBackgroundResource", background(e.phase, kind == Kind.BAR))
    v.setTextViewText(R.id.ezan_arabic, e.nextArabic)
    v.setVisibility(R.id.ezan_count, View.VISIBLE)
    // оставащото време: Chronometer брой до часа на молитвата сам
    val base = SystemClock.elapsedRealtime() + (e.nextAt - now)
    v.setChronometer(R.id.ezan_count, base, null, true)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) v.setChronometerCountDown(R.id.ezan_count, true)
}

```

```

when (kind) {
    Kind.NEXT -> {
        v.setTextViewText(R.id.ezan_place, e.place)
        v.setTextViewText(R.id.ezan_name, e.nextName)
        v.setTextViewText(R.id.ezan_at, e.shortAt)
        v.setVisibility(R.id.ezan_at, View.VISIBLE)
        v.setTextViewText(R.id.ezan_until, e.untilText)
        v.setVisibility(R.id.ezan_until, View.VISIBLE)
        v.setVisibility(R.id.ezan_dome, View.VISIBLE)
    }
    Kind.DAY -> {
        v.setTextViewText(R.id.ezan_place, e.place)
        v.setTextViewText(R.id.ezan_label, "${e.nextLabel} . ")
        v.setTextViewText(R.id.ezan_name, e.nextName)
        v.setVisibility(R.id.ezan_label_row, View.VISIBLE)
        v.setTextViewText(R.id.ezan_at, e.atText)
        v.setVisibility(R.id.ezan_at, View.VISIBLE)
        v.setVisibility(R.id.ezan_rows, View.VISIBLE)
        v.setVisibility(R.id.ezan_empty, View.GONE)
        for ((i, ids) in ROW_IDS.withIndex()) {
            val r = e.rows.getOrNull(i)
            if (r == null) {
                v.setVisibility(ids[0], View.GONE)
                continue
            }
            v.setVisibility(ids[0], View.VISIBLE)
            val strong = r.state == "now" || r.state == "next"
            val color = when (r.state) {
                "now" -> GOLD
                "past" -> PAST
                "next" -> TEXT
                else -> LATER
            }
            v.setTextViewText(ids[1], styled(r.name, strong))
            v.setTextViewText(ids[2], styled(r.time, strong))
            v.setTextColor(ids[1], color)
            v.setTextColor(ids[2], color)
            v.setInt(ids[0], "setBackgroundResource", if (r.state == "now") R.drawable.ezan_widget_row_now
else 0)
        }
    }
    Kind.BAR -> {
        v.setTextViewText(R.id.ezan_place, e.place)
        v.setVisibility(R.id.ezan_place, View.VISIBLE)
        v.setTextViewText(R.id.ezan_name, e.nextName)
        v.setTextViewText(R.id.ezan_at, "${e.atText}")
        v.setVisibility(R.id.ezan_at, View.VISIBLE)
    }
}

/** Няма кадри (приложението не е отворено или не е отворено 7 дни): „Отвори Езан...“. */
private fun renderEmpty(context: Context, v: RemoteViews, kind: Kind, emptyText: String?) {
    val text = emptyText ?: context.getString(R.string.ezan_widget_empty)
    v.setInt(R.id.ezan_widget_root, "setBackgroundResource", if (kind == Kind.BAR)
R.drawable.ezan_widget_bg_empty_h else R.drawable.ezan_widget_bg_empty)
    v.setTextViewText(R.id.ezan_arabic, "إِذَان")
    v.setVisibility(R.id.ezan_count, View.GONE)
    v.setChronometer(R.id.ezan_count, SystemClock.elapsedRealtime(), null, false)
    when (kind) {
        Kind.NEXT -> {

```

```

        v.setTextViewText(R.id.ezan_place, "")
        v.setTextViewText(R.id.ezan_name, text)
        v.setVisibility(R.id.ezan_at, View.GONE)
        v.setVisibility(R.id.ezan_until, View.GONE)
        v.setVisibility(R.id.ezan_dome, View.GONE)
    }
    Kind.DAY -> {
        v.setTextViewText(R.id.ezan_place, "")
        v.setVisibility(R.id.ezan_label_row, View.GONE)
        v.setVisibility(R.id.ezan_at, View.GONE)
        v.setVisibility(R.id.ezan_rows, View.GONE)
        v.setVisibility(R.id.ezan_empty, View.VISIBLE)
        v.setTextViewText(R.id.ezan_empty, text)
    }
    Kind.BAR -> {
        v.setVisibility(R.id.ezan_place, View.GONE)
        v.setTextViewText(R.id.ezan_name, text)
        v.setVisibility(R.id.ezan_at, View.GONE)
    }
}

private fun styled(text: String, bold: Boolean): CharSequence {
    if (!bold) return text
    val s = SpannableString(text)
    s.setSpan(StyleSpan(Typeface.BOLD), 0, text.length, Spanned.SPAN_EXCLUSIVE_EXCLUSIVE)
    return s
}

/** Фонът: градиентът на текущата молитва (вертикален; за „Лента“ – хоризонтален). */
private fun background(phase: String, horizontal: Boolean): Int =
    if (horizontal) {
        when (phase) {
            "fajr" -> R.drawable.ezan_widget_bg_fajr_h
            "sunrise" -> R.drawable.ezan_widget_bg_sunrise_h
            "dhuhr" -> R.drawable.ezan_widget_bg_dhuhr_h
            "asr" -> R.drawable.ezan_widget_bg_asr_h
            "maghrib" -> R.drawable.ezan_widget_bg_maghrib_h
            "isha" -> R.drawable.ezan_widget_bg_isha_h
            else -> R.drawable.ezan_widget_bg_empty_h
        }
    } else {
        when (phase) {
            "fajr" -> R.drawable.ezan_widget_bg_fajr
            "sunrise" -> R.drawable.ezan_widget_bg_sunrise
            "dhuhr" -> R.drawable.ezan_widget_bg_dhuhr
            "asr" -> R.drawable.ezan_widget_bg_asr
            "maghrib" -> R.drawable.ezan_widget_bg_maghrib
            "isha" -> R.drawable.ezan_widget_bg_isha
            else -> R.drawable.ezan_widget_bg_empty
        }
    }
}

private fun launchIntent(context: Context): PendingIntent? {
    val intent = context.packageManager.getLaunchIntentForPackage(context.packageName) ?: return null
    intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_RESET_TASK_IF_NEEDED
    return PendingIntent.getActivity(
        context,
        OPEN_REQUEST,
        intent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
    )
}

```

```

    )
}

/* ----- следващото обновяване */

private fun refreshIntent(context: Context): PendingIntent =
    PendingIntent.getBroadcast(
        context,
        REFRESH_REQUEST,
        Intent(context, WidgetRefreshReceiver::class.java),
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
    )

/**
 * Планира обновяване в часа на следващата смяна (молитва или полунощ).
 * RTC – не буди телефона: ако спи, обновяването идва, щом се събуди (тогава го и виждаш).
 * Без widget-и на екрана – нищо не се планира.
 */
fun scheduleNext(context: Context) {
    val am = context.getSystemService(Context.ALARM_SERVICE) as? AlarmManager ?: return
    val pi = refreshIntent(context)
    am.cancel(pi)
    if (count(context) == 0) return
    val at = WidgetStore.nextChange(WidgetStore.load(context), System.currentTimeMillis()) ?: return
    try {
        if (AlarmScheduler.canScheduleExact(context)) {
            am.setExact(AlarmManager.RTC, at, pi)
        } else {
            // без „Аларми и напомняния“ – в прозорец от минута (Android 12+ го разширява до ~10 мин.);
            // set() може да закъснее с часове
            am.setWindow(AlarmManager.RTC, at, 60_000L, pi)
        }
    } catch (e: SecurityException) {
        am.setWindow(AlarmManager.RTC, at, 60_000L, pi)
    }
}

/** Общото за трите widget-a: системата вика onUpdate при поставяне и на всеки час (резерва). */
abstract class PrayerWidgetProvider(private val kind: PrayerWidgets.Kind) : AppWidgetProvider() {
    override fun onUpdate(context: Context, manager: AppWidgetManager, ids: IntArray) {
        try {
            PrayerWidgets.update(context, manager, ids, kind)
        } catch (e: Exception) {
            Log.w("AdhanWidget", "onUpdate: ${e.message}")
        }
        PrayerWidgets.scheduleNext(context)
    }

    override fun onDisabled(context: Context) {
        // последният widget от този вид е махнат – ако няма други, scheduleNext спира обновяването
        PrayerWidgets.scheduleNext(context)
    }
}

/** „Следваща“ 2×2. */
class NextPrayerWidget : PrayerWidgetProvider(PrayerWidgets.Kind.NEXT)

/** „Денят“ 4×2. */
class DayWidget : PrayerWidgetProvider(PrayerWidgets.Kind.DAY)

```

```

/** „Лента“ 4x1. */
class BarWidget : PrayerWidgetProvider(PrayerWidgets.Kind.BAR)

/** В часа на всяка молитва и в полунощ – новият кадър. */
class WidgetRefreshReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        PrayerWidgets.updateAll(context)
    }
}

```

AdhanNativeModule.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- `setWidgetData` и `getWidgetCount`.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AdhanNativeModule.kt

198 реда

```

package expo.modules.adhannative

import android.app.AlarmManager
import android.app.NotificationManager
import android.content.ActivityNotFoundException
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.PowerManager
import android.provider.Settings
import expo.modules.kotlin.Promise
import expo.modules.kotlin.exception.Exceptions
import expo.modules.kotlin.modules.Module
import expo.modules.kotlin.modules.ModuleDefinition

/**
 * Native модуль на Езан (само Android):
 * - етап 4: точни известия (Android 12+);
 * - етап 5: алармата с пълния езан (AlarmManager.setAlarmClock + foreground service +
 * екран „Аларма“), аларма на цял екран (Android 14+), работа на заден план
 * (оптимизация на батерията) и местоположение без Google услуги (Huawei);
 * - етап 6: звуците; етап 8: widget-ите (PrayerWidgets.kt).
 */
class AdhanNativeModule : Module() {
    private val context: Context
    get() = appContext.reactContext ?: throw Exceptions.ReactContextLost()

    private val packageUri: Uri
    get() = Uri.parse("package:${context.packageName}")

    private fun open(intent: Intent, fallback: Intent? = null) {
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        try {
            context.startActivity(intent)
        } catch (e: ActivityNotFoundException) {
            val f = fallback ?: Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
            context.startActivity(f.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK))
        }
    }

    override fun definition() = ModuleDefinition {
        Name("AdhanNative")
    }
}

```

```

/* ----- точни аларми (етап 4) */

// Може ли приложението да планира точни аларми. До Android 12 – винаги да.
// От Android 14 разрешението „Аларми и напомняния“ е изключено по подразбиране.
Function("canScheduleExactAlarms") {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.S) {
        true
    } else {
        val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
        alarmManager.canScheduleExactAlarms()
    }
}

// Отваря системния екран „Аларми и напомняния“ за Езан.
// Ако производителят го е махнал – екрана с информация за приложението.
Function("openExactAlarmSettings") {
    val intent =
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM, packageUri)
        } else {
            Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
        }
    open(intent)
}

/* ----- алармата с езана (етап 5) */

// Заменя всички аларми (JSON масив, виж AlarmData). Връща колко са планирани.
Function("setAlarms") { json: String ->
    AlarmScheduler.setAlarms(context, json)
}

// Запазеният списък (JSON) – за „Проверка на известията“.
Function("getAlarms") {
    AlarmScheduler.storedJson(context)
}

// Последните звъннали аларми (JSON): планиран и истински час.
Function("getAlarmHistory") {
    AlarmScheduler.historyJson(context)
}

// Пробна аларма след delayMs.
Function("testAlarm") { json: String, delayMs: Double ->
    AlarmScheduler.scheduleTest(context, json, delayMs.toLong())
}

Function("stopAlarm") {
    AlarmService.command(context, AlarmService.ACTION_STOP)
}

/* ----- звуци (етап 6) */

// Преслушване: вграден звук (res/raw) или свой файл (file://...).
Function("previewSound") { source: String ->
    SoundPreview.play(context, source)
}

Function("stopPreview") {
    SoundPreview.stop()
}

```

```

// Свой кратък звук: откъс до maxSec (срез на пауза, заглъхване) в Notifications/Ezan.
AsyncFunction("prepareShortSound") { uri: String, title: String, maxSec: Double ->
    val r = ShortSound.prepare(context, uri, title, maxSec)
    mapOf(
        "uri" to r.uri,
        "duration" to r.duration,
        "originalDuration" to r.originalDuration,
        "trimmed" to r.trimmed,
    )
}

Function("deleteShortSound") { uri: String ->
    ShortSound.delete(context, uri)
}

// Канал за известие със свой звук (content://...).
Function("createSoundChannel") { id: String, name: String, uri: String, alarm: Boolean, vibrate:
Boolean, pattern: List<Double> ->
    ShortSound.createChannel(context, id, name, uri, alarm, vibrate, pattern.map { it.toLong()
}.toLongArray())
}

// Дължината на звуков файл в секунди; -1 – не е звук или не може да се прочете.
AsyncFunction("getAudioDuration") { uri: String ->
    SoundSource.durationSec(context, uri)
}

/* ----- аларма на цял екран (Android 14+) */

Function("canUseFullScreenIntent") {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        true
    } else {
        val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        nm.canUseFullScreenIntent()
    }
}

Function("openFullScreenIntentSettings") {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        open(Intent(Settings.ACTION_MANAGE_APP_USE_FULL_SCREEN_INTENT, packageUri))
    } else {
        open(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri))
    }
}

/* ----- работа на заден план */

// true – телефонът не ограничава Езан (оптимизацията на батерията е изключена за него).
Function("isIgnoringBatteryOptimizations") {
    val pm = context.getSystemService(Context.POWER_SERVICE) as PowerManager
    pm.isIgnoringBatteryOptimizations(context.packageName)
}

// Настройките на Езан в телефона (оттам: Батерия → Без ограничения).
Function("openAppSettings") {
    open(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri))
}

// Списъкът „Оптимизация на батерията“ на Android.

```

```

Function("openBatteryOptimizationSettings") {
    open(Intent(Settings.ACTION_IGNORE_BATTERY_OPTIMIZATION_SETTINGS))
}

/* ----- widget-и (етап 8) */

// Кадрите на widget-ите (JSON от src/domain/widget.ts): записва ги и прерисува widget-ите.
Function("setWidgetData") { json: String ->
    PrayerWidgets.setData(context, json)
}

// Колко widget-а на Езан има на началния екран.
Function("getWidgetCount") {
    PrayerWidgets.count(context)
}

/* ----- местоположение без Google (Huawei) */

Function("hasGoogleServices") {
    NativeLocation.hasGoogleServices(context)
}

AsyncFunction("getCurrentLocation") { timeoutMs: Double, promise: Promise ->
    val ctx = context
    if (!NativeLocation.hasPermission(ctx)) {
        promise.resolve(null)
        return@AsyncFunction
    }
    NativeLocation.current(ctx, timeoutMs.toLong()) { location ->
        promise.resolve(location?.let { NativeLocation.toMap(it) })
    }
}
}
}
}

```

AlarmReceiver.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- BootReceiver: след рестарт, обновяване и смяна на часа/часовата зона – и widget-ите наново (Chronometer-ът брои от времето от включването).

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmReceiver.kt

79 реда

```

package expo.modules.adhannative

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.os.Build
import android.os.PowerManager
import android.util.Log

/** Часът на молитвата: пуска AlarmService (звук, вибрация, известие, екран „Аларма“). */
class AlarmReceiver : BroadcastReceiver() {
    companion object {
        private var wakeLock: PowerManager.WakeLock? = null

        /**
         * Будилникът държи телефона буден само докато върви onReceive. Докато услугата
         * тръгне и пусне звука, процесорът не бива да заспи – затова собствен wake lock
         * (най-много 60 сек.); услугата го пуска, щом звукът тръгне.
         */
    }
}

```

```

    */
    @Synchronized
    fun acquireWakeLock(context: Context) {
        if (wakeLock?.isHeld == true) return
        val pm = context.getSystemService(Context.POWER_SERVICE) as PowerManager
        wakeLock = pm.newWakeLock(PowerManager.PARTIAL_WAKE_LOCK, "adhan:alarm").apply {
            setReferenceCounted(false)
            acquire(60_000L)
        }
    }

    @Synchronized
    fun releaseWakeLock() {
        try {
            wakeLock?.let { if (it.isHeld) it.release() }
        } catch (e: Exception) {
            // нищо
        }
        wakeLock = null
    }
}

override fun onReceive(context: Context, intent: Intent) {
    val json = intent.getStringExtra(AlarmScheduler.EXTRA_ALARM) ?: return
    val alarm = AlarmData.fromJson(json) ?: return
    // телефонът е бил изключен и алармата идва много по-късно - не звъни
    if (System.currentTimeMillis() - alarm.at > AlarmScheduler.MAX_LATE_MS) return
    acquireWakeLock(context)

    val service = Intent(context, AlarmService::class.java)
        .setAction(AlarmService.ACTION_START)
        .putExtra(AlarmScheduler.EXTRA_ALARM, json)
    try {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            context.startForegroundService(service)
        } else {
            context.startService(service)
        }
    } catch (e: Exception) {
        Log.w("AdhanAlarm", "start service: ${e.message}")
        releaseWakeLock()
    }
}

}

/** След рестарт, обновяване на приложението или смяна на часа - алармите се планират наново. */
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        try {
            AlarmScheduler.rescheduleAll(context)
        } catch (e: Exception) {
            Log.w("AdhanAlarm", "reschedule: ${e.message}")
        }
        // widget-ите (етап 8): след рестарт/смяна на часа Chronometer-ът трябва нова основа
        try {
            PrayerWidgets.updateAll(context)
        } catch (e: Exception) {
            Log.w("AdhanWidget", "boot: ${e.message}")
        }
    }
}
}

```

AndroidManifest.xml modules/adhan-native/android/src/main

- Трите widget-а и WidgetRefreshReceiver.

modules/adhan-native/android/src/main/AndroidManifest.xml

90 реда

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Алармата с пълния езан (етап 5) -->
  <uses-permission android:name="android.permission.WAKE_LOCK" />
  <uses-permission android:name="android.permission.VIBRATE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
  <uses-permission android:name="android.permission.USE_FULL_SCREEN_INTENT" />
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

  <!-- Android 11+: за да види дали на телефона има Google Play Services (Huawei без Google) -->
  <queries>
    <package android:name="com.google.android.gms" />
  </queries>

  <application>
    <!-- Екранът „Аларма“ върху заключенния екран; отделна задача, за да не вадя цялото приложение -->
    <activity
      android:name="expo.modules.adhannative.AlarmActivity"
      android:exported="false"
      android:showWhenLocked="true"
      android:turnScreenOn="true"
      android:excludeFromRecents="true"
      android:launchMode="singleInstance"
      android:taskAffinity="${applicationId}.alarm"
      android:theme="@android:style/Theme.Material.NoActionBar" />

    <!-- Пуска езана и вибрацията, докато свърши (или „Спри“) -->
    <service
      android:name="expo.modules.adhannative.AlarmService"
      android:exported="false"
      android:foregroundServiceType="mediaPlayback" />

    <!-- Будилникът на Android (setAlarmClock) вика този receiver в часа на молитвата -->
    <receiver
      android:name="expo.modules.adhannative.AlarmReceiver"
      android:exported="false" />

    <!-- След рестарт, обновяване на приложението, смяна на часа или на разрешението – алармите наново -->
    <receiver
      android:name="expo.modules.adhannative.BootReceiver"
      android:exported="true">
      <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
        <action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
        <action android:name="android.intent.action.TIME_SET" />
        <action android:name="android.intent.action.TIMEZONE_CHANGED" />
        <action android:name="android.app.action.SCHEDULE_EXACT_ALARM_PERMISSION_STATE_CHANGED" />
      </intent-filter>
    </receiver>

    <!-- Widget-ите (етап 8): „Следваща“ 2×2, „Денят“ 4×2, „Лента“ 4×1 -->
    <receiver
      android:name="expo.modules.adhannative.NextPrayerWidget"
      android:label="@string/ezan_widget_next_name"
      android:exported="false">
      <intent-filter>
```

```

        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/ezan_widget_next_info" />
</receiver>
<receiver
    android:name="expo.modules.adhannative.DayWidget"
    android:label="@string/ezan_widget_day_name"
    android:exported="false">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/ezan_widget_day_info" />
</receiver>
<receiver
    android:name="expo.modules.adhannative.BarWidget"
    android:label="@string/ezan_widget_bar_name"
    android:exported="false">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/ezan_widget_bar_info" />
</receiver>

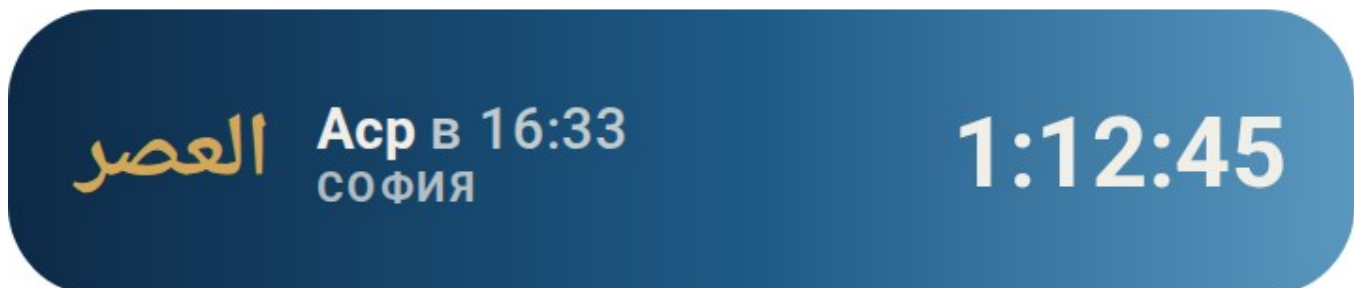
<!-- Новият кадър на widget-ите в часа на всяка молитва и в полунощ -->
<receiver
    android:name="expo.modules.adhannative.WidgetRefreshReceiver"
    android:exported="false" />
</application>
</manifest>

```

Стъпка 5

Android – изгледите

RemoteViews позволява само прости изгледи (LinearLayout, FrameLayout, TextView, ImageView, Chronometer). Свой шрифтове не зарежда – затова са системните (Roboto), удебелени като в mockup-а.



Прегледите в списъка с widget-и (на английски и на български)

ezan_widget_next.xml modules/adhan-native/android/src/main/res/layout

- „Следваща“ 2×2.

modules/adhan-native/android/src/main/res/layout/ezan_widget_next.xml

118 реда

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Шрифтовете са системните: widget-ите не могат да зареждат свои шрифтове (RemoteViews). -->
<!-- Widget „Следваща“ 2×2 (етап 8) – по тоскур-а. Текстовете ги задава PrayerWidgets.kt. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/ezan_widget_root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/ezan_widget_bg_dhuhr"
    android:clipToOutline="true">

    <ImageView
        android:id="@+id/ezan_dome"
```

```

android:layout_width="130dp"
android:layout_height="65dp"
android:layout_gravity="bottom|end"
android:layout_marginEnd="-10dp"
android:layout_marginBottom="-2dp"
android:alpha="0.22"
android:importantForAccessibility="no"
android:src="@drawable/ezan_widget_dome" />

```

<LinearLayout

```

android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:paddingStart="16dp"
android:paddingEnd="16dp"
android:paddingTop="12dp"
android:paddingBottom="14dp">

```

<LinearLayout

```

android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal"
android:baselineAligned="true">

```

<TextView

```

    android:id="@+id/ezan_arabic"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:textStyle="bold"
    android:textSize="24sp"
    android:textColor="#D4A857"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="العصر"
    android:layout_weight="1" />

```

<TextView

```

    android:id="@+id/ezan_place"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:textSize="9.5sp"
    android:textColor="#C7F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_place"
    android:letterSpacing="0.1" />

```

</LinearLayout>

<LinearLayout

```

android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal">

```

<TextView

```

    android:id="@+id/ezan_name"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:textSize="17sp"
    android:textColor="#F2EFE8"
    android:ellipsize="end"

```

```

        android:text="@string/ezan_widget_sample_name"
        android:maxLines="2" />
<TextView
    android:id="@+id/ezan_at"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif-medium"
    android:textSize="17sp"
    android:textColor="#C7F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_at"
    android:layout_marginStart="5dp"
    android:fontFeatureSettings="tnum" />
</LinearLayout>

<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />

<Chronometer
    android:id="@+id/ezan_count"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:fontFeatureSettings="tnum"
    android:letterSpacing="-0.02"
    android:includeFontPadding="false"
    android:textSize="31sp"
    android:textColor="#F2EFE8"
    android:singleLine="true" />
<TextView
    android:id="@+id/ezan_until"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif-medium"
    android:textSize="11.5sp"
    android:textColor="#C7F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_until" />
</LinearLayout>
</FrameLayout>

```

ezan_widget_day.xml modules/adhan-native/android/src/main/res/layout

- „Денят“ 4×2 – лявата колона и картата с шестте часа (по тегла, за да се побира на всеки телефон).

modules/adhan-native/android/src/main/res/layout/ezan_widget_day.xml

317 реда

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Шрифтовете са системните: widget-ите не могат да зареждат свои шрифтове (RemoteViews). -->
<!-- Widget „Денят“ 4×2 (етап 8) – по тоскуп-а. Текстовете ги задава PrayerWidgets.kt. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/ezan_widget_root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/ezan_widget_bg_dhuhr"

```

```

android:clipToOutline="true">

<ImageView
    android:layout_width="150dp"
    android:layout_height="75dp"
    android:layout_gravity="bottom|start"
    android:layout_marginStart="6dp"
    android:layout_marginBottom="-2dp"
    android:alpha="0.22"
    android:importantForAccessibility="no"
    android:src="@drawable/ezan_widget_dome" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal"
    android:paddingStart="16dp"
    android:paddingEnd="14dp"
    android:paddingTop="14dp"
    android:paddingBottom="12dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="46"
        android:orientation="vertical">
        <TextView
            android:id="@+id/ezan_place"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:textStyle="bold"
            android:textSize="10.5sp"
            android:textColor="#B8F2EFE8"
            android:maxLines="1"
            android:ellipsize="end"
            android:text="@string/ezan_widget_sample_place"
            android:letterSpacing="0.1" />
        <FrameLayout
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1" />
        <TextView
            android:id="@+id/ezan_arabic"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textStyle="bold"
            android:textSize="26sp"
            android:textColor="#D4A857"
            android:maxLines="1"
            android:ellipsize="end"
            android:text="العصر" />
    <LinearLayout
        android:id="@+id/ezan_label_row"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:id="@+id/ezan_label"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"

```

```

        android:fontFamily="sans-serif-medium"
        android:textSize="13sp"
        android:textColor="#C7F2EFE8"
        android:maxLines="1"
        android:ellipsize="end"
        android:text="@string/ezan_widget_sample_label" />
<TextView
    android:id="@+id/ezan_name"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:textSize="13sp"
    android:textColor="#F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_name" />
</LinearLayout>
<Chronometer
    android:id="@+id/ezan_count"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:fontFeatureSettings="tnum"
    android:letterSpacing="-0.02"
    android:includeFontPadding="false"
    android:textSize="30sp"
    android:textColor="#F2EFE8"
    android:singleLine="true" />
<TextView
    android:id="@+id/ezan_at"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif-medium"
    android:textSize="12sp"
    android:textColor="#B8F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_at_long"
    android:fontFeatureSettings="tnum" />
</LinearLayout>

<FrameLayout
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_weight="54"
    android:layout_marginStart="14dp">

    <LinearLayout
        android:id="@+id/ezan_rows"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:background="@drawable/ezan_widget_panel"
        android:paddingStart="6dp"
        android:paddingEnd="6dp"
        android:paddingTop="5dp"
        android:paddingBottom="5dp">
        <LinearLayout
            android:id="@+id/ezan_row0"

```

```

        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="8dp"
        android:paddingEnd="8dp">
        <TextView
            android:id="@+id/ezan_row0_name"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:fontFamily="sans-serif"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1"
            android:ellipsize="end" />
        <TextView
            android:id="@+id/ezan_row0_time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:fontFeatureSettings="tnum"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1" />
    </LinearLayout>
    <LinearLayout
        android:id="@+id/ezan_row1"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="8dp"
        android:paddingEnd="8dp">
        <TextView
            android:id="@+id/ezan_row1_name"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:fontFamily="sans-serif"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1"
            android:ellipsize="end" />
        <TextView
            android:id="@+id/ezan_row1_time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:fontFeatureSettings="tnum"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1" />
    </LinearLayout>
    <LinearLayout
        android:id="@+id/ezan_row2"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"

```

```

        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="8dp"
        android:paddingEnd="8dp">
        <TextView
            android:id="@+id/ezan_row2_name"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:fontFamily="sans-serif"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1"
            android:ellipsize="end" />
        <TextView
            android:id="@+id/ezan_row2_time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:fontFeatureSettings="tnum"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1" />
    </LinearLayout>
    <LinearLayout
        android:id="@+id/ezan_row3"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="8dp"
        android:paddingEnd="8dp">
        <TextView
            android:id="@+id/ezan_row3_name"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:fontFamily="sans-serif"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1"
            android:ellipsize="end" />
        <TextView
            android:id="@+id/ezan_row3_time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:fontFeatureSettings="tnum"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1" />
    </LinearLayout>
    <LinearLayout
        android:id="@+id/ezan_row4"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="8dp"

```

```

        android:paddingEnd="8dp">
        <TextView
            android:id="@+id/ezan_row4_name"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:fontFamily="sans-serif"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1"
            android:ellipsize="end" />
        <TextView
            android:id="@+id/ezan_row4_time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:fontFeatureSettings="tnum"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1" />
    </LinearLayout>
    <LinearLayout
        android:id="@+id/ezan_row5"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="8dp"
        android:paddingEnd="8dp">
        <TextView
            android:id="@+id/ezan_row5_name"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:fontFamily="sans-serif"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1"
            android:ellipsize="end" />
        <TextView
            android:id="@+id/ezan_row5_time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif"
            android:fontFeatureSettings="tnum"
            android:textSize="12.5sp"
            android:textColor="#D9F2EFE8"
            android:maxLines="1" />
    </LinearLayout>
</LinearLayout>

    <TextView
        android:id="@+id/ezan_empty"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center_vertical"
        android:fontFamily="sans-serif"
        android:textStyle="bold"
        android:textSize="13sp"
        android:textColor="#C7F2EFE8"

```

```

        android:visibility="gone" />
    </FrameLayout>
</LinearLayout>
</FrameLayout>

```

ezan_widget_bar.xml modules/adhan-native/android/src/main/res/layout

- „Лента“ 4×1.

modules/adhan-native/android/src/main/res/layout/ezan_widget_bar.xml

88 реда

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Шрифтовете са системните: widget-ите не могат да зареждат свои шрифтове (RemoteViews). -->
<!-- Widget „Лента“ 4×1 (етап 8) – по моцкуп-а. Текстовете ги задава PrayerWidgets.kt. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/ezan_widget_root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/ezan_widget_bg_dhuhr_h"
    android:clipToOutline="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="18dp"
        android:paddingEnd="18dp">

        <TextView
            android:id="@+id/ezan_arabic"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textStyle="bold"
            android:textSize="24sp"
            android:textColor="#D4A857"
            android:maxLines="1"
            android:ellipsize="end"
            android:text="العصر" />

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:layout_marginStart="12dp"
            android:layout_marginEnd="8dp"
            android:orientation="vertical">

            <LinearLayout
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:orientation="horizontal">

                <TextView
                    android:id="@+id/ezan_name"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:fontFamily="sans-serif"
                    android:textStyle="bold"
                    android:textSize="15sp"
                    android:textColor="#F2EFE8"
                    android:ellipsize="end"
                    android:text="@string/ezan_widget_sample_name"
                    android:maxLines="2" />
            </LinearLayout>
        </LinearLayout>
    </FrameLayout>

```

```

<TextView
    android:id="@+id/ezan_at"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif-medium"
    android:textSize="15sp"
    android:textColor="#C7F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_at_long"
    android:fontFeatureSettings="tnum" />
</LinearLayout>
<TextView
    android:id="@+id/ezan_place"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:textSize="11sp"
    android:textColor="#B8F2EFE8"
    android:maxLines="1"
    android:ellipsize="end"
    android:text="@string/ezan_widget_sample_place"
    android:letterSpacing="0.08" />
</LinearLayout>
<Chronometer
    android:id="@+id/ezan_count"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="sans-serif"
    android:textStyle="bold"
    android:fontFeatureSettings="tnum"
    android:letterSpacing="-0.02"
    android:includeFontPadding="false"
    android:textSize="26sp"
    android:textColor="#F2EFE8"
    android:singleLine="true" />
</LinearLayout>
</FrameLayout>

```

ezan_widget_next_info.xml modules/adhan-native/android/src/main/res/xml

- Размерът (2×2), прегледът, описанието; `updatePeriodMillis` 1 ч. – резерва. Другите два (`_day_info` , `_bar_info`) са същите с 4×2 и 4×1.

modules/adhan-native/android/src/main/res/xml/ezan_widget_next_info.xml

14 реда

```

<?xml version="1.0" encoding="utf-8"?>
<appwidget-provider xmlns:android="http://schemas.android.com/apk/res/android"
    android:minWidth="110dp"
    android:minHeight="110dp"
    android:minResizeWidth="110dp"
    android:minResizeHeight="110dp"
    android:targetCellWidth="2"
    android:targetCellHeight="2"
    android:resizeMode="horizontal|vertical"
    android:updatePeriodMillis="3600000"
    android:initialLayout="@layout/ezan_widget_next"
    android:previewImage="@drawable/ezan_widget_preview_next"

```

```
android:description="@string/ezan_widget_next_desc"
android:widgetCategory="home_screen" />
```

ezan_widget_bg_dhuhr.xml modules/adhan-native/android/src/main/res/drawable

- Фонът – градиентът на молитвата, по един за всяка молитва (+ `_h` – хоризонтален за „Лента“).

modules/adhan-native/android/src/main/res/drawable/ezan_widget_bg_dhuhr.xml

12 реда

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Фонът на widget-a: градиентът на молитвата (src/theme/gradients.ts) -->
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape="rectangle">
  <gradient
    android:type="linear"
    android:angle="270"
    android:startColor="#0D2A47"
    android:centerColor="#1E5A86"
    android:centerY="0.55"
    android:endColor="#5B98C0" />
  <corners android:radius="@dimen/ezan_widget_radius" />
</shape>
```

ezan_widget_strings.xml modules/adhan-native/android/src/main/res/values-bg

- Имената и описанията в списъка с widget-и (и `values/` – на английски).

modules/adhan-native/android/src/main/res/values-bg/ezan_widget_strings.xml

16 реда

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string name="ezan_widget_next_name">Следваща молитва</string>
  <string name="ezan_widget_next_desc">Следващата молитва и колко остава</string>
  <string name="ezan_widget_day_name">Часовете за деня</string>
  <string name="ezan_widget_day_desc">Следващата молитва, колко остава и всички часове за деня</string>
  <string name="ezan_widget_bar_name">Лента</string>
  <string name="ezan_widget_bar_desc">Следващата молитва и колко остава на един ред</string>
  <string name="ezan_widget_empty">Отвори Езан, за да се покажат часовете.</string>
  <string name="ezan_widget_sample_place">СОФИЯ</string>
  <string name="ezan_widget_sample_name">Асп</string>
  <string name="ezan_widget_sample_at">16:33</string>
  <string name="ezan_widget_sample_at_long">В 16:33</string>
  <string name="ezan_widget_sample_until">остава до Асп</string>
  <string name="ezan_widget_sample_label">Следваща · </string>
</resources>
```

Стъпка 6

iPhone – SwiftUI

Кодът на widget-а върви в разширението на WidgetKit – отделно от приложението.

PrayerWidget.tsx src/widgets

- Функциите с `'widget'` се превеждат в отделен пакет. В тях не може да се ползва нищо отвън – всичко идва от кадъра (props).
- `systemSmall` / `systemMedium` – като на Android; фонът с `containerBackground` (градиент), оставащото време – `Text timerInterval`.

- Заклучен екран: `accessoryInline` (ред над часа), `accessoryCircular` (дъга до следващата молитва), `accessoryRectangular`; вторият widget – `PrayerArabicWidget` (кръгъл, арабското име).
- Не е проверено на iPhone – при първия iOS build.

src/widgets/PrayerWidget.tsx

223 реда

```
import {
  AccessoryWidgetBackground,
  HStack,
  ProgressView,
  Spacer,
  Text,
  VStack,
  ZStack,
} from '@expo/ui/swift-ui';
import {
  background,
  containerBackground,
  font,
  foregroundStyle,
  frame,
  kerning,
  lineLimit,
  minimumScaleFactor,
  monospacedDigit,
  opacity,
  padding,
  progressViewStyle,
  shapes,
  widgetURL,
} from '@expo/ui/swift-ui/modifiers';
import { createWidget, type WidgetEnvironment } from 'expo-widgets';

import type { WidgetEntry } from '@domain/widget';

/**
 * Widget-ите за iPhone (етап 8) – по одобрения тоскуп.
 *
 * Функциите с 'widget' се превеждат в отделен пакет, който върви в разширението на WidgetKit:
 * в тях не може да се ползва нищо извън функцията – всичко идва от props (кадъра от
 * src/domain/widget.ts) и от environment. Компонентите и модификаторите са тези на @expo/ui.
 * Оставащото време е Text с таймер – брои го iOS, без приложението.
 */

export type PrayerWidgetProps = WidgetEntry | { empty: true; emptyText: string };

const PrayerWidget = (props: PrayerWidgetProps, environment: WidgetEnvironment) => {
  'widget';
  const GOLD = '#D4A857';
  const TEXT = '#F2EFE8';
  const DIM = 'rgba(242,239,232,0.78)';
  const family = environment.widgetFamily;
  const lock = family === 'accessoryCircular' || family === 'accessoryRectangular' || family ===
'accessoryInline';

  if ('empty' in props) {
    return (
      <VStack
        alignment="leading"
        spacing={6}

```

```

        modifiers={[
          containerBackground({ type: 'linearGradient', colors: ['#0F1B2D', '#1A2A44'], startPoint: { x:
0.5, y: 0 }, endPoint: { x: 0.5, y: 1 } }, 'widget'),
          widgetURL('adhan://'),
        ]}
      >
      {lock ? null : <Text modifiers={[font({ size: 26, weight: 'bold' })],
foregroundStyle(GOLD)]}>ذان</Text>
      <Text modifiers={[font({ size: lock ? 12 : 13, weight: 'semibold' })], foregroundStyle(lock ?
'#FFFFFF' : TEXT)]>
        {props.emptyText}
      </Text>
    </VStack>
  );
}

const e = props;
const timer = { lower: new Date(e.prevAt), upper: new Date(e.nextAt) };
const bg = containerBackground(
  { type: 'linearGradient', colors: [...e.colors], startPoint: { x: 0.5, y: 0 }, endPoint: { x: 0.5, y: 1
} },
  'widget',
);
const url = widgetURL('adhan://');

/* ----- заключен экран */
if (family === 'accessoryInline') {
  return <Text modifiers={[url]}>`${e.nextName} ${e.shortAt}`</Text>;
}

if (family === 'accessoryCircular') {
  return (
    <ZStack modifiers={[url, containerBackground('#00000000', 'widget')]}>
      <AccessoryWidgetBackground />
      <ProgressView timerInterval={timer} countsDown={false} modifiers={[progressViewStyle('circular')]}
    />

    <VStack spacing={0}>
      <Text modifiers={[font({ size: 10, weight: 'bold' })], lineLimit(1), minimumScaleFactor(0.6)]>
        {e.nextName.toLocaleUpperCase()}
      </Text>
      <Text modifiers={[font({ size: 15, weight: 'heavy' })], monospacedDigit()}>{e.nextTime}</Text>
    </VStack>
  </ZStack>
);
}

if (family === 'accessoryRectangular') {
  return (
    <VStack alignment="leading" spacing={0} modifiers={[url, containerBackground('#00000000', 'widget'),
frame({ maxWidth: 10000, alignment: 'leading' })]}>
      <HStack spacing={6} alignment="firstTextBaseline">
        <Text modifiers={[font({ size: 15, weight: 'heavy' })]}>`${e.nextName} ${e.nextTime}`</Text>
        <Text modifiers={[font({ size: 15, weight: 'bold' })]}>{e.nextArabic}</Text>
      </HStack>
      <Text timerInterval={timer} countsDown modifiers={[font({ size: 20, weight: 'heavy' })],
monospacedDigit()} />
      <Text modifiers={[font({ size: 11, weight: 'semibold' }), opacity(0.7), lineLimit(1)]>
        {`${e.leftLabel} · ${e.place.charAt(0)}${e.place.slice(1).toLocaleLowerCase()}`}
      </Text>
    </VStack>
  );
}

```

```

}

/* ----- начален екран */
const nextBlock = (
  <VStack alignment="leading" spacing={0}>
    <Text modifiers={[font({ size: 26, weight: 'bold' }), foregroundStyle(GOLD)]}>{e.nextArabic}</Text>
    <HStack spacing={0}>
      <Text modifiers={[font({ size: 13, weight: 'semibold' }), foregroundStyle(DIM)]}>`$${e.nextLabel} .`</Text>
      <Text modifiers={[font({ size: 13, weight: 'heavy' }), foregroundStyle(TEXT)]}>{e.nextName}</Text>
    </HStack>
    <Text
      timerInterval={timer}
      countsDown
      modifiers={[font({ size: 30, weight: 'heavy' }), monospacedDigit(), foregroundStyle(TEXT),
lineLimit(1), minimumScaleFactor(0.7)]}
    />
    <Text modifiers={[font({ size: 12, weight: 'medium' }), foregroundStyle(DIM), monospacedDigit()]}>
{e.atText}</Text>
  </VStack>
);

if (family === 'systemMedium') {
  return (
    <HStack spacing={14} alignment="top" modifiers={[bg, url]}>
      <VStack alignment="leading" spacing={0} modifiers={[frame({ width: 146, alignment: 'leading' })]}>
        <Text modifiers={[font({ size: 10.5, weight: 'bold' }), kerning(1), foregroundStyle(DIM)]}>
{e.place}</Text>
        <Spacer />
        {nextBlock}
      </VStack>
      <VStack
        spacing={1}
        modifiers={[
          padding({ vertical: 5, horizontal: 6 }),
          background('rgba(6,10,20,0.32)', shapes.roundedRectangle({ cornerRadius: 14 })),
        ]}
      >
        {e.rows.map((r) => (
          <HStack
            key={r.id}
            modifiers={[
              padding({ vertical: 1, horizontal: 8 }),
              background(r.state === 'now' ? 'rgba(212,168,87,0.16)' : '#00000000',
shapes.roundedRectangle({ cornerRadius: 8 })),
            ]}
          >
            <Text
              modifiers={[
                font({ size: 12.5, weight: r.state === 'now' || r.state === 'next' ? 'heavy' : r.state
=== 'past' ? 'medium' : 'semibold' }),
                foregroundStyle(r.state === 'now' ? GOLD : r.state === 'past' ? 'rgba(242,239,232,0.45)'
: r.state === 'next' ? TEXT : 'rgba(242,239,232,0.85)'),
              ]}
            >
              {r.name}
            </Text>
            <Spacer />
            <Text
              modifiers={[
                font({ size: 12.5, weight: r.state === 'now' || r.state === 'next' ? 'heavy' : r.state

```

```

=== 'past' ? 'medium' : 'semibold' })),
    monospacedDigit(),
    foregroundStyle(r.state === 'now' ? GOLD : r.state === 'past' ? 'rgba(242,239,232,0.45)'
: r.state === 'next' ? TEXT : 'rgba(242,239,232,0.85)'),
  ]}
  >
  {r.time}
</Text>
</HStack>
  )}}
</VStack>
</HStack>
  );
}

// systemSmall
return (
  <VStack alignment="leading" spacing={0} modifiers={[bg, url]}>
    <HStack alignment="firstTextBaseline">
      <Text modifiers={[font({ size: 24, weight: 'bold' }), foregroundStyle(GOLD)]}>{e.nextArabic}</Text>
      <Spacer />
      <Text modifiers={[font({ size: 9.5, weight: 'bold' }), kerning(1), foregroundStyle(DIM),
lineLimit(1)]}>{e.pPlace}</Text>
    </HStack>
    <HStack spacing={5} alignment="firstTextBaseline">
      <Text modifiers={[font({ size: 17, weight: 'heavy' }), foregroundStyle(TEXT)]}>{e.nextName}</Text>
      <Text modifiers={[font({ size: 17, weight: 'medium' }), foregroundStyle(DIM), monospacedDigit(),
lineLimit(1), minimumScaleFactor(0.7)]}>
        {e.shortAt}
      </Text>
    </HStack>
    <Spacer />
    <Text
      timerInterval={timer}
      countsDown
      modifiers={[font({ size: 31, weight: 'heavy' }), monospacedDigit(), foregroundStyle(TEXT),
lineLimit(1), minimumScaleFactor(0.7)]}
    />
    <Text modifiers={[font({ size: 11.5, weight: 'medium' }), foregroundStyle(DIM), lineLimit(1)]}>
{e.untilText}</Text>
  </VStack>
  );
};

/** Вторият кръгъл widget за заключения екран: арабското име и оставащото време. */
const PrayerArabicWidget = (props: PrayerWidgetProps, environment: WidgetEnvironment) => {
  'widget';
  if ('empty' in props) {
    return <Text modifiers={[font({ size: 18, weight: 'bold' }), containerBackground('#00000000',
'widget')]}>اذان</Text>;
  }
  const e = props;
  return (
    <ZStack modifiers={[widgetURL('adhan://'), containerBackground('#00000000', 'widget')]}>
      <AccessoryWidgetBackground />
      <VStack spacing={0}>
        <Text modifiers={[font({ size: 17, weight: 'bold' }), lineLimit(1), minimumScaleFactor(0.6)]}>
{e.nextArabic}</Text>
        <Text
          timerInterval={{ lower: new Date(e.prevAt), upper: new Date(e.nextAt) }}
          countsDown

```

```

        modifiers={[font({ size: 12, weight: 'heavy' }), monospacedDigit(), lineLimit(1),
minimumScaleFactor(0.6), padding({ horizontal: 6 })]}
      />
    </VStack>
  </ZStack>
);
};

export const prayerWidget = createWidget<PrayerWidgetProps>('PrayerWidget', PrayerWidget);
export const prayerArabicWidget = createWidget<PrayerWidgetProps>('PrayerArabicWidget',
PrayerArabicWidget);

```

app.json корен на проекта

- expo-widgets – двата widget-а и размерите им ("android": null – на Android са нашите).

app.json

170 реда

```

{
  "expo": {
    "name": "Adhan",
    "slug": "adhan",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "dark",
    "ios": {
      "supportsTablet": false,
      "bundleIdentifier": "com.ilkoadamov.adhan",
      "infoPlist": {
        "CFBundleAllowMixedLocalizations": true
      }
    },
    "android": {
      "adaptiveIcon": {
        "backgroundColor": "#0F1B2D",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false,
      "package": "com.ilkoadamov.adhan",
      "permissions": [
        "android.permission.SCHEDULE_EXACT_ALARM"
      ],
      "blockedPermissions": [
        "android.permission.READ_EXTERNAL_STORAGE",
        "android.permission.WRITE_EXTERNAL_STORAGE",
        "android.permission.RECORD_AUDIO"
      ]
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
      "expo-router",
      [
        "expo-splash-screen",
        {
          "backgroundColor": "#0F1B2D",

```

```

    "image": "./assets/splash-icon.png",
    "imageWidth": 160
  },
],
"expo-font",
[
  "expo-build-properties",
  {
    "android": {
      "cmakeVersion": "3.31.6"
    }
  }
],
[
  "expo-localization",
  {
    "supportedLocales": {
      "ios": [
        "en",
        "bg"
      ],
      "android": [
        "en",
        "bg"
      ]
    }
  }
],
[
  "expo-location",
  {
    "locationWhenInUsePermission": "Adhan uses your location to show the prayer times for the place
where you are.",
    "locationAlwaysAndWhenInUsePermission": false,
    "locationAlwaysPermission": false,
    "motionUsagePermission": false
  }
],
"expo-background-task",
[
  "expo-notifications",
  {
    "icon": "./assets/notification-icon.png",
    "color": "#D4A857",
    "sounds": [
      "./assets/sounds/ezan_chime.wav",
      "./assets/sounds/chime_soft.wav",
      "./assets/sounds/takbir_makkah.wav",
      "./assets/sounds/takbir_madinah.wav",
      "./assets/sounds/takbir_aqsa.wav",
      "./assets/sounds/takbir_istanbul.wav",
      "./assets/sounds/notify_airport.wav",
      "./assets/sounds/notify_popup.wav",
      "./assets/sounds/notify_right.wav",
      "./assets/sounds/notify_modern.wav",
      "./assets/sounds/notify_magical.wav",
      "./assets/sounds/notify_chime_bright.wav",
      "./assets/sounds/notify_doorbell.wav",
      "./assets/sounds/notify_celestial.wav",
      "./assets/sounds/notify_bell_ding.wav",
      "./assets/sounds/notify_crisp_ding.wav",
    ]
  }
]

```

```

        "./assets/sounds/notify_shimmer.wav",
        "./assets/sounds/notify_smoke_alarm.wav",
        "./assets/sounds/notify_swoop.wav",
        "./assets/sounds/notify_retro.wav",
        "./assets/sounds/notify_robot.wav",
        "./assets/sounds/notify_three_beep.wav",
        "./assets/sounds/notify_machine_beep.wav",
        "./assets/sounds/notify_water_drop.wav",
        "./assets/sounds/notify_hang_up.wav",
        "./assets/sounds/notify_standard.wav"
    ]
}
],
[
    "expo-audio",
    {
        "microphonePermission": false,
        "recordAudioAndroid": false,
        "enableBackgroundPlayback": false,
        "enableBackgroundRecording": false
    }
],
"./plugins/withFullSounds",
[
    "expo-widgets",
    {
        "widgets": [
            {
                "name": "PrayerWidget",
                "displayName": "Adhan",
                "description": "The next prayer and the time left",
                "ios": {
                    "supportedFamilies": [
                        "systemSmall",
                        "systemMedium",
                        "accessoryCircular",
                        "accessoryRectangular",
                        "accessoryInline"
                    ]
                },
                "android": null
            },
            {
                "name": "PrayerArabicWidget",
                "displayName": "Adhan · Arabic",
                "description": "The next prayer in Arabic and the time left",
                "ios": {
                    "supportedFamilies": [
                        "accessoryCircular"
                    ]
                },
                "android": null
            }
        ]
    }
],
"scheme": "adhan",
"backgroundColor": "#0F1B2D",
"experiments": {
    "typedRoutes": true
}

```

```

    },
    "locales": {
      "bg": "./locales/bg.json",
      "en": "./locales/en.json"
    }
  }
}

```

Стъпка 7

Тестове

226 теста общо, 8 нови.

widget.test.ts src/domain/__tests__

- 15:20 – Зухр, следва Аср след 1:12:45 (като в москуп-а); след Иша – Фаджр „утре“; английски текстове.
- Кадрите: от началото на текущата молитва, нов в часа на всяка молитва и в полунощ, 7 дни напред; еднакви през целия интервал на молитвата.

src/domain/__tests__/widget.test.ts

97 реда

```

import { bg } from '@i18n/strings';
import { PHASE_GRADIENTS } from '@theme/gradients';

import { locationFromCoords } from '../resolve';
import { planWidget, widgetEntryAt, WIDGET_DAYS, type WidgetInput } from '../widget';

const sofia = locationFromCoords(42.6977, 23.3219);
const options = { location: sofia, method: 'Turkey' as const, madhab: 'shafi' as const };
const texts = { prayers: bg.prayers, ...bg.hero, ...bg.widget };

const input = (now: Date): WidgetInput => ({ now, options, placeName: 'София', texts, gradients:
PHASE_GRADIENTS });
const at = (h: number, m: number, s = 0, day = 5) => new Date(2026, 9, day, h, m, s);

describe('widget - кадърът в даден момент', () => {
  it('15:20 - Зухр, следва Аср в 16:33 (като в москуп-а)', () => {
    const e = widgetEntryAt(at(15, 20, 15), input(at(15, 20, 15)));
    expect(e.phase).toBe('dhuhr');
    expect(e.colors).toEqual(PHASE_GRADIENTS.dhuhr);
    expect(e.place).toBe('СОФИЯ');
    expect([e.next, e.nextName, e.nextArabic, e.nextTime]).toEqual(['asr', 'Аср', '16:33', 'العصر']);
    expect(e.atText).toBe('в 16:33');
    expect(e.shortAt).toBe('16:33');
    expect(e.untilText).toBe('остават до Аср');
    expect(e.nextAt - at(15, 20, 15).getTime()).toBe(((1 * 60 + 12) * 60 + 45) * 1000);
    expect(e.prevAt).toBe(at(13, 20).getTime());
    expect(e.rows.map((r) => `${r.name} ${r.time} ${r.state}`)).toEqual([
      'Фаджр 05:53 past',
      'Изгрев 07:20 past',
      'Зухр 13:20 now',
      'Аср 16:33 next',
      'Магриб 19:11 later',
      'Иша 20:32 later',
    ]);
  });

  it('след Иша - следващият е Фаджр утре', () => {

```

```

const e = widgetEntryAt(at(23, 15), input(at(23, 15)));
expect(e.phase).toBe('isha');
expect(e.next).toBe('fajr');
expect(e.atText).toMatch(/^утре в 05:5\d$/);
expect(e.shortAt).toMatch(/^утре 05:5\d$/);
expect(e.rows.filter((r) => r.state === 'next')).toHaveLength(0);
expect(e.rows[5].state).toBe('now');
});

it('английски текстове', () => {
  const en = { prayers: { fajr: 'Fajr', sunrise: 'Sunrise', dhuhr: 'Dhuhr', asr: 'Asr', maghrib:
'Maghrib', isha: 'Isha' } };
  const e = widgetEntryAt(at(21, 10), {
    ...input(at(21, 10)),
    placeName: 'Sofia',
    texts: { ...texts, prayers: en.prayers, next: 'Next', at: (t) => `at ${t}`, tomorrowAt: (t) =>
'tomorrow at ${t}`, tomorrowShort: (t) => `tomorrow ${t}`, until: (n) => `until ${n}`, left: 'left' },
  });
  expect(e.place).toBe('SOFIA');
  expect(e.nextName).toBe('Fajr');
  expect(e.atText).toMatch(/^tomorrow at 05:5\d$/);
});
});

describe('widget - кадрите за следващите дни', () => {
  const plan = planWidget(input(at(15, 20, 15)));

  it('първият кадър започва от началото на текущата молитва', () => {
    expect(plan[0].from).toBe(at(13, 20).getTime());
    expect(plan[0].next).toBe('asr');
  });

  it('нов кадър в часа на всяка молитва и в полунощ, по ред', () => {
    expect(plan[1].from).toBe(at(16, 33).getTime());
    expect(plan[1].phase).toBe('asr');
    expect(plan.some((e) => e.from === at(0, 0, 0, 6).getTime())).toBe(true);
    for (let i = 1; i < plan.length; i++) expect(plan[i].from).toBeGreaterThan(plan[i - 1].from);
    // кадърът важи, докато дойде следващата молитва
    for (const e of plan) expect(e.nextAt).toBeGreaterThan(e.from);
  });

  it('стига до 7 дни напред: 6 молитви + полунощ на ден', () => {
    const last = plan[plan.length - 1];
    expect(last.from).toBeLessThanOrEqual(at(0, 0, 0, 5 + WIDGET_DAYS).getTime());
    expect(plan.length).toBeGreaterThanOrEqual(WIDGET_DAYS * 7 - 2);
    expect(plan.length).toBeLessThanOrEqual(WIDGET_DAYS * 7 + 2);
  });

  it('в полунощ денят в списъка се сменя', () => {
    const midnight = plan.find((e) => e.from === at(0, 0, 0, 6).getTime());
    expect(midnight.phase).toBe('isha');
    expect(midnight.next).toBe('fajr');
    expect(midnight.rows.find((r) => r.id === 'fajr')!.state).toBe('next');
    expect(midnight.rows.filter((r) => r.state === 'past')).toHaveLength(0);
  });

  it('един и същ резултат през целия интервал - няма излишни обновявания', () => {
    const later = planWidget(input(at(16, 0)));
    expect(JSON.stringify(later)).toBe(JSON.stringify(plan));
    const afterAsr = planWidget(input(at(16, 40)));
    expect(afterAsr[0].from).toBe(at(16, 33).getTime());
  });

```

```
});  
});
```

Трябва да видиш: Tests: 226 passed, 226 total

```
npm test
```

Стъпка 8

Проверка

На телефона:

- Иконата на началния екран и в списъка с приложения – злато „I“ върху нощно синьо.
- Задръж на празно място → **Widgets** → Adhan: трите widget-а с прегледи. Сложи ги.
- Оставащото време тече всяка секунда; в часа на молитвата widget-ът сменя молитвата и фона (може да провериш с ръчна корекция от Настройки → Корекции, за да дойде молитвата след минута).
- Смени мястото в приложението → widget-ите се обновяват веднага.
- Рестартирай телефона → widget-ите са верни и броят.
- Тап върху widget-а отваря приложението.

Терминал

```
git add .  
git commit -m "Етап 8: widget-и и нова икона"  
git tag stage-8  
git push  
git push origin stage-8
```